

Types of Knowledge in AI

1. Declarative Knowledge (Descriptive Knowledge)

Declarative knowledge consists of facts and information about the world that AI systems store and retrieve when needed. It represents "what" is known rather than "how" to do something. **This type of knowledge is often stored in structured formats like databases, ontologies, and knowledge graphs.**

For example, a fact such as "Paris is the capital of France" is declarative knowledge. AI applications like search engines and virtual assistants use this type of knowledge to answer factual queries and provide relevant information.

2. Procedural Knowledge (How-To Knowledge)

Procedural knowledge **defines the steps or methods required to perform specific tasks**. It represents **"how" to accomplish something rather than just stating a fact.**

For instance, knowing how to solve a quadratic equation or how to drive a car falls under procedural knowledge. AI systems, such as expert systems and robotics, utilize procedural knowledge to execute tasks that require sequences of actions. This type of knowledge is often encoded in rule-based systems, decision trees, and machine learning models.

3. Meta-Knowledge (Knowledge About Knowledge)

Refers to knowledge about **how information is structured, used, and validated**. It helps AI determine the **reliability, relevance, and applicability of knowledge in different scenarios**.

For example, an AI system deciding whether a piece of medical advice comes from a trusted scientific source or a random blog post is using meta-knowledge. This type of knowledge is crucial in AI models for filtering misinformation, optimizing learning strategies, and improving decision-making.

4. Heuristic Knowledge (Experience-Based Knowledge)

Heuristic knowledge is **derived from experience**, intuition, and trial-and-error methods. It allows AI systems to make educated guesses or approximate solutions when exact answers are difficult to compute.

For example, a navigation system suggesting an alternate route based on past traffic patterns is applying heuristic knowledge. AI search algorithms, such as A* search and genetic algorithms, leverage heuristics to optimize problem-solving processes, making decisions more efficient in real-world scenarios.

Introduction to Propositional Logic

[Propositional logic](#), also known as propositional calculus or Boolean logic, is a simple and fundamental form of logic. It deals with propositions, which are statements that can be either true or false. The basic components of propositional logic include:

- **Propositions:** Basic statements that are either true or false.
- **Logical Connectives:** Operators such as AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$), and BICONDITIONAL ($\leftrightarrow$) used to combine propositions.
- **Truth Values:** Each proposition has a truth value of either true (T) or false (F).

Example

Consider the propositions:

- P: "It is raining."
- Q: "The ground is wet."

Using logical connectives, we can form complex expressions like $P \rightarrow Q$ (If it is raining, then the ground is wet).

Summary Table for Propositional Logic Connectives

Connective	Symbol	Name	Description	Example	Truth Table
AND	$\wedge$	Conjunction	True if both propositions are true	$P \wedge Q$	P
OR	$\vee$	Disjunction	True if at least one of the propositions is true	$P \vee Q$	P
NOT	$\neg$	Negation	True if the proposition is false	$\neg P$	P
IMPLIES	$\rightarrow$	Implication	True if the first proposition implies the second proposition	$P \rightarrow Q$	P
BICONDITIONAL	$\leftrightarrow$	Biconditional	True if both propositions are either true or false	$P \leftrightarrow Q$	P

Introduction to First-Order Logic

[First-order logic \(FOL\)](#), also known as predicate logic or first-order predicate calculus, extends [propositional logic](#) by introducing quantifiers and predicates. It allows for a more expressive representation of knowledge by dealing with objects, properties, and relationships.

The basic components of first-order logic include:

- **Constants:** Specific objects in the domain (e.g., Alice, Bob).
- **Variables:** Symbols that can represent any object in the domain (e.g., x, y).
- **Predicates:** Functions that map objects to truth values (e.g., Likes(Alice, IceCream)).
- **Quantifiers:** Symbols that indicate the scope of a statement (e.g., $\forall$ (forall), $\exists$ (exists)).
- **Logical Connectives:** Same as in propositional logic.

Example

Consider the predicates:

- Likes(x,y): "x likes y."

Using quantifiers, we can express statements like $\forall x \exists y (\text{Likes}(x,y))$ (For every person x, there exists a person y such that x likes y).

Summary Table for First-Order Logic Components

Component	Symbol	Name	Description	Example
Universal Quantifier	$\forall$	For All	Asserts that a predicate is true for all elements in the domain	$\forall x (P(x))$
Existential Quantifier	$\exists$	There Exists	Asserts that there is at least one element in the domain for which the predicate is true	$\exists x (P(x))$
Predicate	$P(x)$	Predicate	A function that returns true or false based on the object(s) it is applied to	$P(x)$: "x is a person"
Conjunction	$\wedge$	AND	True if both predicates are true	$P(x) \wedge Q(x)$
Disjunction	$\vee$	OR	True if at least one of the predicates is true	$P(x) \vee Q(x)$
Negation	$\neg$	NOT	True if the predicate is false	$\neg P(x)$
Implication	$\rightarrow$	IMPLIES	True if the first predicate implies the second predicate	$P(x) \rightarrow Q(x)$

Component	Symbol	Name	Description	Example
Biconditional	$\leftrightarrow$	BICONDITIONAL	True if both predicates are either true or false	$P(x) \leftrightarrow Q(x)$

Key Differences Between Propositional Logic and First-Order Logic

Expressiveness

- **Propositional Logic:** Limited to simple true/false statements without the ability to express relationships between objects. Suitable for scenarios where the complexity of relationships is low.
- **First-Order Logic:** More expressive, capable of representing relationships, properties of objects, and quantification. Suitable for complex scenarios involving multiple objects and relationships.

Syntax and Semantics

- **Propositional Logic:** Uses propositions and logical connectives. Each proposition represents a distinct, indivisible truth statement.
- **First-Order Logic:** Uses predicates, constants, variables, and quantifiers in addition to logical connectives. Allows for the construction of more complex statements involving multiple objects and their properties.

Quantification

- **Propositional Logic:** Does not support quantifiers. Statements are either universally true or false.
- **First-Order Logic:** Supports quantifiers ($\forall$ and $\exists$), enabling statements about all or some objects in the domain.

Use Cases

- **Propositional Logic:** Suitable for simple problems like circuit design, troubleshooting, and basic rule-based systems.
- **First-Order Logic:** Suitable for more complex problems involving relationships and properties, such as [natural language processing](#), semantic web, and AI reasoning systems.

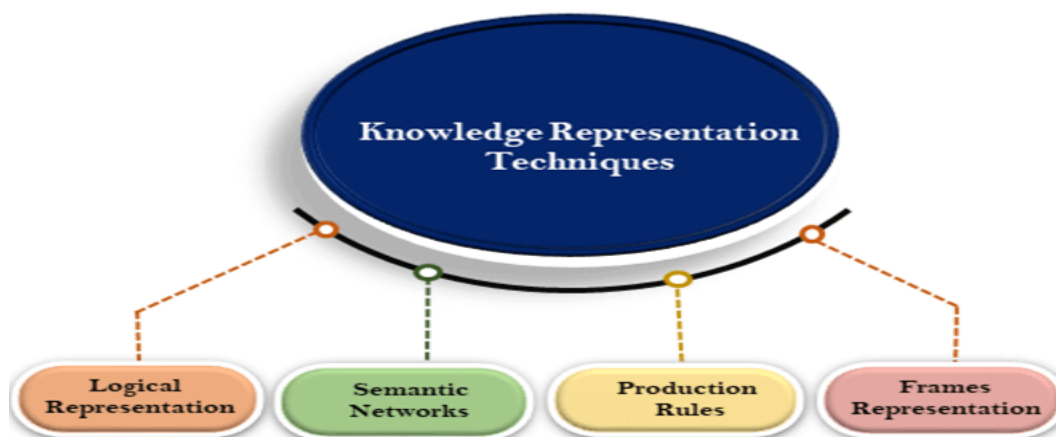
Key Differences Summarized

Feature	Propositional Logic	First-Order Logic
Basic Unit	Propositions	Predicates, constants, variables
Expressiveness	Limited to true/false statements	Expressive, can represent relationships and properties
Quantifiers	None	Universal ($\forall$) and Existential ($\exists$)
Syntax	Combines propositions using logical connectives	Uses predicates and quantifiers
Semantics	Truth tables	Interpretation over a domain
Use Cases	Simple problems (e.g., circuit design, rule-based systems)	Complex problems (e.g., AI reasoning, ontology modeling)
Example	$P \rightarrow Q$	$\forall x \exists y (\text{Likes}(x, y)) \forall x \exists y (\text{Likes}(x, y))$

Techniques of knowledge representation

There are mainly four ways of knowledge representation which are given as follows:

1. Logical Representation
2. Semantic Network Representation
3. Frame Representation
4. Production Rules



1. Logical Representation

Logical representation is a language with some concrete rules which deals with propositions and has no ambiguity in representation. Logical representation means drawing a conclusion based on various conditions. This representation lays down some important communication rules. It consists of precisely defined syntax and semantics which supports the sound inference. Each sentence can be translated into logics using syntax and semantics.

Syntax:

- Syntaxes are the rules which decide how we can construct legal sentences in the logic.
- It determines which symbol we can use in knowledge representation.
- How to write those symbols.

Semantics:

- Semantics are the rules by which we can interpret the sentence in the logic.
- Semantic also involves assigning a meaning to each sentence.

Logical representation can be categorised into mainly two logics:

1. Propositional Logics
2. Predicate logics

Note: We will discuss Propositional Logics and Predicate logics in later chapters.

Advantages of logical representation:

1. Logical representation enables us to do logical reasoning.
2. Logical representation is the basis for the programming languages.

Disadvantages of logical Representation:

1. Logical representations have some restrictions and are challenging to work with.
2. Logical representation technique may not be very natural, and inference may not be so efficient.

Note: Do not be confused with logical representation and logical reasoning as logical representation is a representation language and reasoning is a process of thinking logically.

2. Semantic Network Representation

Semantic networks are alternative of predicate logic for knowledge representation. In Semantic networks, we can represent our knowledge in the form of graphical networks. This network consists of nodes representing objects and arcs which describe the relationship between those

objects. Semantic networks can categorize the object in different forms and can also link those objects. Semantic networks are easy to understand and can be easily extended.

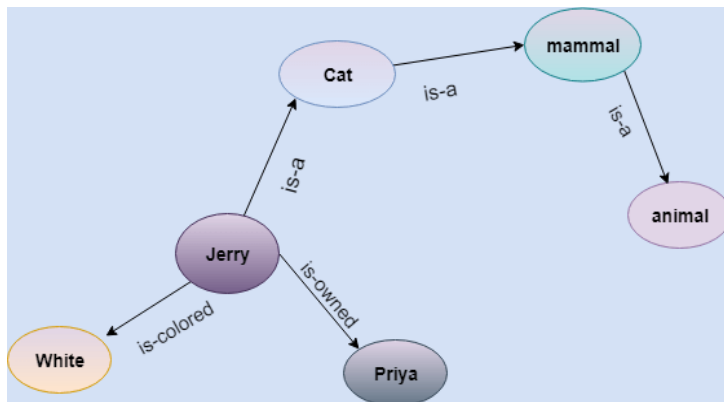
This representation consist of mainly two types of relations:

1. IS-A relation (Inheritance)
2. Kind-of-relation

Example: Following are some statements which we need to represent in the form of nodes and arcs.

Statements:

1. Jerry is a cat.
2. Jerry is a mammal
3. Jerry is owned by Priya.
4. Jerry is brown colored.
5. All Mammals are animal.



In the above diagram, we have represented the different type of knowledge in the form of nodes and arcs. Each object is connected with another object by some relation.

Drawbacks in Semantic representation:

1. Semantic networks take more computational time at runtime as we need to traverse the complete network tree to answer some questions. It might be possible in the worst case scenario that after traversing the entire tree, we find that the solution does not exist in this network.
2. Semantic networks try to model human-like memory (Which has 1015 neurons and links) to store the information, but in practice, it is not possible to build such a vast semantic network.

3. These types of representations are inadequate as they do not have any equivalent quantifier, e.g., for all, for some, none, etc.
4. Semantic networks do not have any standard definition for the link names.
5. These networks are not intelligent and depend on the creator of the system.

Advantages of Semantic network:

1. Semantic networks are a natural representation of knowledge.
2. Semantic networks convey meaning in a transparent manner.
3. These networks are simple and easily understandable.

3. Frame Representation

A frame is a record like structure which consists of a collection of attributes and its values to describe an entity in the world. Frames are the AI data structure which divides knowledge into substructures by representing stereotypes situations. It consists of a collection of slots and slot values. These slots may be of any type and sizes. Slots have names and values which are called facets.

Facets: The various aspects of a slot is known as **Facets**. Facets are features of frames which enable us to put constraints on the frames. Example: IF-NEEDED facts are called when data of any particular slot is needed. A frame may consist of any number of slots, and a slot may include any number of facets and facets may have any number of values. A frame is also known as **slot-filter knowledge representation** in artificial intelligence.

Frames are derived from semantic networks and later evolved into our modern-day classes and objects. A single frame is not much useful. Frames system consist of a collection of frames which are connected. In the frame, knowledge about an object or event can be stored together in the knowledge base. The frame is a type of technology which is widely used in various applications including Natural language processing and machine visions.

Example: 1

Let's take an example of a frame for a book

Slots	Filters
Title	Artificial Intelligence
Genre	Computer Science
Author	Peter Norvig
Edition	Third Edition

Year 1996

Page 1152

Example 2:

Let's suppose we are taking an entity, Peter. Peter is an engineer as a profession, and his age is 25, he lives in city London, and the country is England. So following is the frame representation for this:

Slots	Filter
Name	Peter
Profession	Doctor
Age	25
Marital status	Single
Weight	78

Advantages of frame representation:

1. The frame knowledge representation makes the programming easier by grouping the related data.
2. The frame representation is comparably flexible and used by many applications in AI.
3. It is very easy to add slots for new attribute and relations.
4. It is easy to include default data and to search for missing values.
5. Frame representation is easy to understand and visualize.

Disadvantages of frame representation:

1. In frame system inference mechanism is not be easily processed.
2. Inference mechanism cannot be smoothly proceeded by frame representation.
3. Frame representation has a much generalized approach.

4. Production Rules

Production rules system consist of (**condition, action**) pairs which mean, "If condition then action". It has mainly three parts:

- The set of production rules

- Working Memory
- The recognize-act-cycle

In production rules agent checks for the condition and if the condition exists then production rule fires and corresponding action is carried out. The condition part of the rule determines which rule may be applied to a problem. And the action part carries out the associated problem-solving steps. This complete process is called a recognize-act cycle.

The working memory contains the description of the current state of problems-solving and rule can write knowledge to the working memory. This knowledge match and may fire other rules.

If there is a new situation (state) generates, then multiple production rules will be fired together, this is called conflict set. In this situation, the agent needs to select a rule from these sets, and it is called a conflict resolution.

Example:

- IF (at bus stop AND bus arrives) THEN action (get into the bus)
- IF (on the bus AND paid AND empty seat) THEN action (sit down).
- IF (on bus AND unpaid) THEN action (pay charges).
- IF (bus arrives at destination) THEN action (get down from the bus).

Advantages of Production rule:

1. The production rules are expressed in natural language.
2. The production rules are highly modular, so we can easily remove, add or modify an individual rule.

Disadvantages of Production rule:

1. Production rule system does not exhibit any learning capabilities, as it does not store the result of the problem for the future uses.
2. During the execution of the program, many rules may be active hence rule-based production systems are inefficient.

Ontological Engineering in Artificial Intelligence

Introduction

Artificial Intelligence, commonly referred to as AI, is now a joint facet of numerous technologies, including natural language processing, autonomous systems, data analysis, and decision-making technologies. **Ontological engineering** involves the generation, modification, and control of ontology, which is a well-defined formalization of a concept within a domain and the relations existing among the concepts. **Ontological engineering** is mainly about the development of a standard and social ontology of a specific domain that can be used to facilitate

understanding across people and other tech systems. This essay will briefly discuss the concepts of ontological engineering, its importance to AI, approaches to creating ontologies, the issues faced in ontological engineering and the utilization of ontologies in various fields.

The Role of Ontologies in AI

One of the most significant difficulties in AI is knowledge representation. To enable machines to reason, that is, to replicate human thought in accomplishing specific tasks, they require a means of storing and using information about the world. The ontologies are actually vocabularies for the specifications of the terms and relationships between them that AI systems can use and comprehend information. Expert systems, semantic technologies, natural language processing and many other applications require a structured representation.

For instance, NLP ontology could outline the kinship of NLP and other related words or concepts; for example, "dog" is a kind of "animal" while "barks" is an action accomplished by "dog". Because of this ontology, an AI system is in a position to improve common human uses of AI, such as translating between languages, determining the sentiment of a sentence or statement or responding to questions.

Components of Ontological Engineering

Concepts:

These are the essential elements of ontology that define objects or concepts of a given area of discourse. For example, in medical ontology, concepts are disease, symptom, treatment, and patient, amongst others.

Relationships:

Relation defines the kind of connection between two concepts or the manner in which two concepts are linked in a particular context. In the medical example, a relationship might be "causes", which could connect a "disease" to "symptoms", or "treats", which link a "treatment" to "disease".

Properties:

The description of specifications makes it clear the additional information of the concerning concepts in as much as attributes/features are concerned. For instance, the disease might include attributes such as name, severity, the organ affected, and so on.

Axioms:

It isn't an assertive statement like an assertion, which is a declarative statement of a hypothesis. Instead, an axiom is a rule that dictates how the concepts exist and how they ensure the stability of the ontology. They define what must not or should not exist in the knowledge base for it by

specifying conditions that must always hold, including, for example: "always holds that at least one symptom is related to every patient".

Instances:

Examples are icons or instances of conceptions. In our medical ontology, an instance could be a particular patient, his complaints, signs, and diagnosis.

Applications of Ontological Engineering in AI

Ontological engineering can be applied in the vast area of AI, as mentioned above, and all of them could benefit from a proper formal representation of knowledge.

Healthcare:

In the area of health care, ontologies allow for the potentialities of knowledge representation - diseases, symptoms, treatments, and their relationships. This makes it possible for an AI system to participate in the disease diagnosis process and the prescription of therapy as well as decision making. For example, SNOMD CT is one of the most used medical ontologies through which the practice of a clear description of clinical concepts is made.

Autonomous Systems:

Information for actions is retrieved from ontologies if a car is to be self-driven or if robots are to be used. These systems required ontologies for constructing objects, actions, and consequential states with a view to addressing safe operations.

Semantic Search Engines:

Ontologies provide increased value to search engines as they allow the understanding of the semantics behind a query and web content. This is useful so that the search results are more precise and meaningful to the users, hence serving the purpose of enhancing the quality of the experience for the users.

E-commerce:

Generally, in operation, e-commerce ontologies are applied in categorizing products, mapping customer preferences, and analyzing transaction information. This ensures that the 'intelligent' systems, including AI, generate tailored recommendations, control inventory, and explore the use and consumption patterns of the products.

Education and E-Learning:

Ontologies in education contain the knowledge domain, learning objectives, and teaching-learning methods and strategies. These ontologies are practical in advanced AI-driven e-learning

platforms, which deliver educational information to learners that meet their requirements and comprehensibility levels.

Legal and Compliance Systems:

Legal ontologies master the vast network of laws, rules and precedents in order to facilitate AI systems in legal investigation, compliance, and contractual examination. This ensures that legal professionals can easily search through numerous legal texts within a shorter period.

Types of Reasoning in AI

The reasoning is classified into the following types:

1. Deductive Reasoning

Deductive reasoning follows a top-down approach where conclusions are drawn from general principles or premises that are known or assumed to be true. This form of reasoning relies on established facts to infer valid conclusions.

Example: If all humans are mortal, and Socrates is a human, then Socrates is mortal.

Application in AI: Deductive reasoning is often used in expert systems and rule-based AI systems, where knowledge is represented through rules (if-then statements). These systems apply general rules to specific problems to derive solutions or make decisions.

2. Inductive Reasoning

Inductive reasoning is a bottom-up approach that involves drawing general conclusions from specific instances or observations. Unlike deductive reasoning, inductive reasoning generates hypotheses rather than certain conclusions, making it more probabilistic.

Example: If we observe that the sun rises in the east every day, we may infer that the sun will rise in the east tomorrow.

Application in AI: Inductive reasoning is widely used in machine learning algorithms. Models trained on data patterns generalize from the data and use this information to make predictions about new, unseen data.

3. Abductive Reasoning

Abductive reasoning starts with an incomplete set of observations and then seeks the most plausible explanation. It focuses on finding the most likely conclusion based on what is known, rather than seeking an absolute truth.

Example: If a patient has a fever and cough, a doctor might hypothesize that they have the flu, even though other illnesses could cause similar symptoms.

Application in AI: *Abductive reasoning is used in diagnostic systems, such as medical diagnosis tools or fault detection systems, where the goal is to identify the most probable cause of a problem given incomplete data.*

4. Analogical Reasoning

Analogical reasoning compares two situations that share similarities, using knowledge from one domain to solve problems in another. This reasoning allows AI systems to draw parallels between similar scenarios.

Example: If flying a drone is similar to piloting a helicopter, knowledge gained from helicopter control can be applied to drones.

Application in AI: *Analogical reasoning is useful in AI for problem-solving, decision-making, and knowledge transfer, especially in robotics and cognitive systems.*

5. Common Sense Reasoning

Common sense reasoning relies on everyday knowledge and experiences to draw conclusions. It simulates how humans use common sense to handle day-to-day situations, which is often challenging for AI due to the implicit nature of this knowledge.

Example: If it rains, we can expect the ground to get wet, even without explicitly stating it.

Application in AI: *AI systems like conversational agents (e.g., Siri, Alexa) are being developed to incorporate commonsense reasoning to handle more natural and complex user interactions effectively.*

6. Monotonic Reasoning

Monotonic reasoning refers to a form of reasoning where conclusions, once drawn, cannot be reversed, even if new information becomes available. This ensures that conclusions remain consistent regardless of updates to the knowledge base.

Example: The statement “The Sahara is a desert” remains true even if more information about the world’s deserts is introduced.

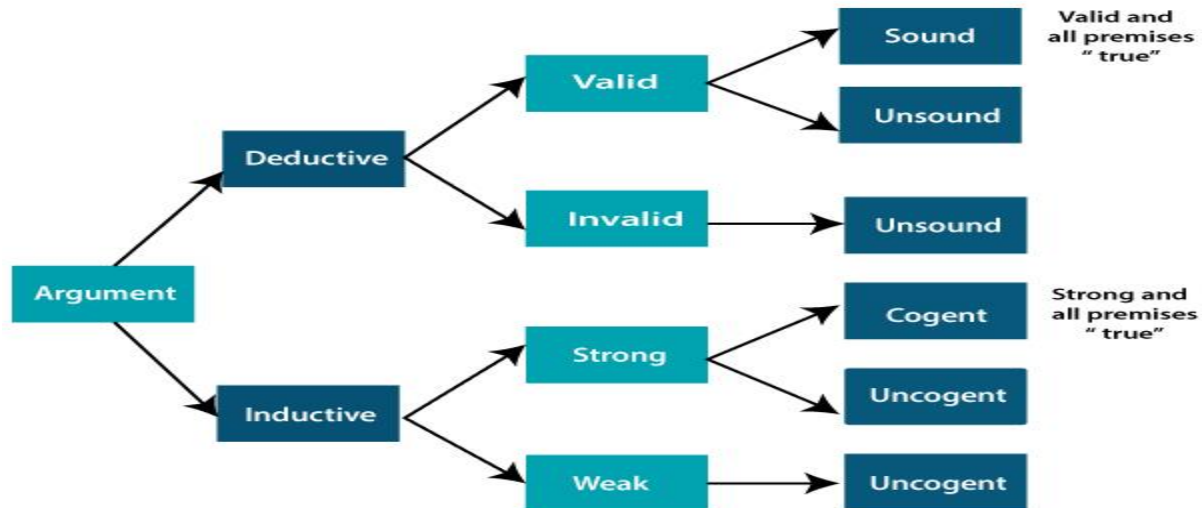
Application in AI: *Monotonic reasoning is applied in conventional reasoning systems and logic-based AI, where consistency is critical. Systems like formal verification tools rely on this type of reasoning to ensure that conclusions do not change over time.*

7. Nonmonotonic Reasoning

In contrast to monotonic reasoning, [nonmonotonic reasoning](#) allows AI systems to revise conclusions when new information becomes available. This is especially useful in dynamic environments where the knowledge base is continuously updated.

Example: Initially concluding that all birds can fly, but revising this conclusion upon learning about penguins, which cannot fly.

Difference between Inductive and Deductive reasoning



Comparison Chart:

Basis for comparison	Deductive Reasoning	Inductive Reasoning
Definition	Deductive reasoning is the form of valid reasoning, to deduce new information or conclusion from known related facts and information.	Inductive reasoning arrives at a conclusion by the process of generalization using specific facts or data.
Approach	Deductive reasoning follows a top-down approach.	Inductive reasoning follows a bottom-up approach.
Starts from	Deductive reasoning starts from Premises.	Inductive reasoning starts from the Conclusion.
Validity	In deductive reasoning conclusion must be true if the premises are true.	In inductive reasoning, the truth of premises does not guarantee the truth of conclusions.
Usage	Use of deductive reasoning is difficult, as we need facts which must be true.	Use of inductive reasoning is fast and easy, as we need evidence instead of true facts. We often use it in our daily life.

Process	Theory→ hypothesis→ patterns→confirmation.	Observations-→patterns→hypothesis→Theory.
Argument	In deductive reasoning, arguments may be valid or invalid.	In inductive reasoning, arguments may be weak or strong.
Structure	Deductive reasoning reaches from general facts to specific facts.	Inductive reasoning reaches from specific facts to general f

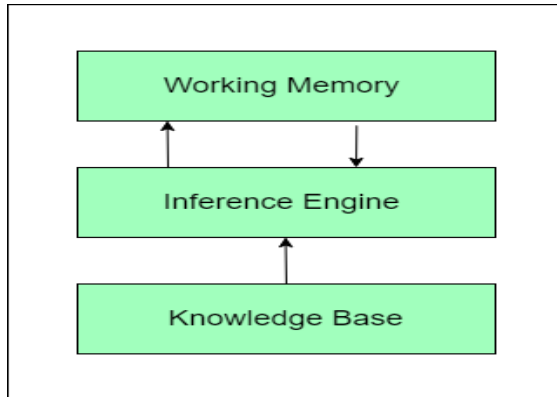
Rule-Based System in AI

Rule-based systems, a foundational technology in artificial intelligence (AI), have long been instrumental in decision-making and problem-solving across various domains. These systems operate on a set of predefined rules and logic to make decisions, perform tasks, or derive conclusions. Despite the rise of more advanced AI methodologies, such as machine learning and neural networks, rule-based systems remain crucial due to their transparency, ease of use, and interpretability.

Components of a Rule-Based System

A typical rule-based system comprises several key components:

1. **Rules:** The core of the system, these are conditional statements that define the system's behavior. A rule generally follows the format "IF condition THEN action." For example, in an expert system for medical diagnosis, a rule might be "IF patient has fever AND cough THEN consider flu."
2. **Knowledge Base:** This is the repository where all the rules and facts are stored. The knowledge base is built from domain-specific knowledge and can be manually curated or derived from expert input.
3. **Inference Engine:** The inference engine is the component that applies the rules to the knowledge base to derive conclusions or make decisions. It interprets the rules, processes them against the current facts or data, and determines the appropriate actions or outputs.
4. **Working Memory:** This is a dynamic component that holds the current facts or data being processed by the system. It is updated as the inference engine applies rules and new information becomes available.
5. **User Interface:** In many rule-based systems, the user interface allows users to interact with the system, input data, and receive outputs or recommendations.



How Rule-Based Systems Work?

The operation of a rule-based system involves several stages:

1. **Data Input:** The system receives input data from the user or another source. This data can range from simple numerical values to complex information like patient symptoms or transaction records.
2. **Rule Matching:** The inference engine examines the input data against the rules stored in the knowledge base. It looks for rules whose conditions match the input data.
3. **Rule Execution:** Once a rule is matched, the inference engine executes the corresponding action. This might involve updating the working memory, deriving new facts, or generating an output.
4. **Conflict Resolution:** In cases where multiple rules are triggered simultaneously, the inference engine uses conflict resolution strategies to determine which rule to apply first. Common strategies include prioritizing rules based on specificity or order of entry.
5. **Output Generation:** The system generates an output based on the executed rules. This output can be a decision, recommendation, or another form of response. For example, in a medical diagnosis system, the output might be a suggested treatment plan.

Example of a Rule-Based System in Action

Consider a simplified example of a rule-based system used in a customer service chatbot:

- **Rule 1:** *If the customer asks about their account balance, then retrieve and display the current balance.*
- **Rule 2:** *If the customer asks about recent transactions, then retrieve and display the last five transactions.*

- **Rule 3:** *If the customer asks to speak to a human agent, then transfer the chat to a human operator.*

When a customer interacts with the chatbot, the inference engine matches their query with the appropriate rule and executes the corresponding action.

Non-monotonic Reasoning

Non-monotonic Reasoning is the process that changes its direction or values as the knowledge base increases.

- It is also known as NMR in Artificial Intelligence.
- Non-monotonic Reasoning will increase or decrease based on the condition.
- Since that Non-monotonic Reasoning depends on assumptions, It will change itself with improving knowledge or facts.
- Example:
 - Consider a bowl of water, If we put it on the stove and turn the flame on it will obviously boil hot and as we will turn off the flame it will cool down gradually.

Probabilistic reasoning in Artificial intelligence

Uncertainty:

Till now, we have learned knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates. With this knowledge representation, we might write $A \rightarrow B$, which means if A is true then B is true, but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called uncertainty.

So to represent uncertain knowledge, where we are not sure about the predicates, we need uncertain reasoning or probabilistic reasoning.

Causes of uncertainty:

Following are some leading causes of uncertainty to occur in the real world.

1. Information occurred from unreliable sources.
2. Experimental Errors
3. Equipment fault
4. Temperature variation
5. Climate change.

Probabilistic reasoning:

Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge. In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.

We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.

In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "It will rain today," "behavior of someone for some situations," "A match between two teams or two players." These are probable sentences for which we can assume that it will happen but not sure about it, so here we use probabilistic reasoning.

Need of probabilistic reasoning in AI:

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

- **Bayes' rule**
- **Bayesian Statistics**

Note: We will learn the above two rules in later chapters.

As probabilistic reasoning uses probability and related terms, so before understanding probabilistic reasoning, let's understand some common terms:

Probability: Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.

1. $0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A .
1. $P(A) = 0$, indicates total uncertainty in an event A .
1. $P(A) = 1$, indicates total certainty in an event A .

We can find the probability of an uncertain event by using the below formula.

$$\text{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- $P(\neg A)$ = probability of a not happening event.
- $P(\neg A) + P(A) = 1$.

Event: Each possible outcome of a variable is called an event.

Sample space: The collection of all possible events is called sample space.

Random variables: Random variables are used to represent the events and objects in the real world.

Prior probability: The prior probability of an event is probability computed before observing new information.

Posterior Probability: The probability that is calculated after all evidence or information has taken into account. It is a combination of prior probability and new information.

Conditional probability:

Conditional probability is a probability of occurring an event when another event has already happened.

Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

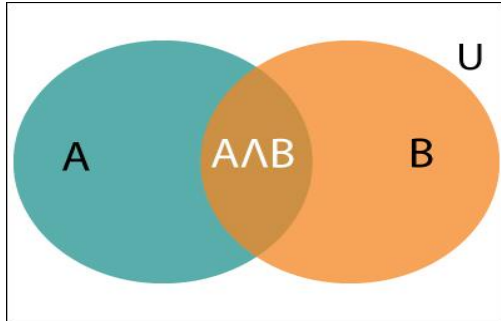
Where $P(A \cap B)$ = Joint probability of a and B

$P(B)$ = Marginal probability of B.

If the probability of A is given and we need to find the probability of B, then it will be given as:

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

It can be explained by using the below Venn diagram, where B is occurred event, so sample space will be reduced to set B, and now we can only calculate event A when event B is already occurred by dividing the probability of $P(A \cap B)$ by $P(B)$.

**Example:**

In a class, there are 70% of the students who like English and 40% of the students who like English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let, A is an event that a student likes Mathematics

B is an event that a student likes English.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics

Basic Understanding of Bayesian Belief Networks

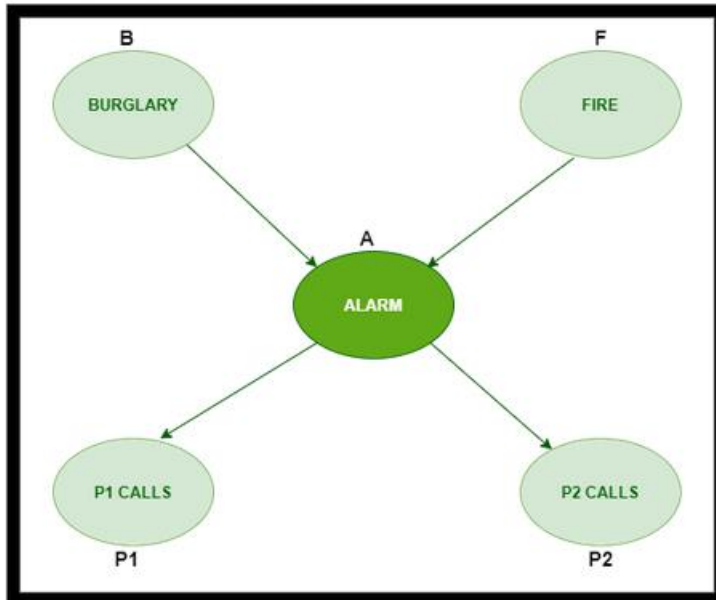
Bayesian Belief Network (BBN) is a graphical model that represents the probabilistic relationships among variables. It is used to handle uncertainty and make predictions or decisions based on probabilities.

- **Graphical Representation:** Variables are represented as nodes in a directed acyclic graph (DAG), and their dependencies are shown as edges.
- **Conditional Probabilities:** Each node's probability depends on its parent nodes, expressed as $P(\text{Variable} | \text{Parent})P(\text{Variable} | \text{Parent})$.
- **Probabilistic Model:** Built from probability distributions, BBNs apply probability theory for tasks like prediction and anomaly detection.

Bayesian Belief Networks are valuable tools for understanding and solving problems involving uncertain events. They are also known as **Bayes networks, belief networks, decision networks, or Bayesian models**.

(Note: A classifier assigns data in a collection to desired categories.)

- Consider this example:



- In the above figure, we have an alarm 'A' – a node, say installed in a house of a person 'gfg', which rings upon two probabilities i.e burglary 'B' and fire 'F', which are – parent nodes of the alarm node. The alarm is the parent node of two probabilities P1 calls 'P1' & P2 calls 'P2' person nodes.
- Upon the instance of burglary and fire, 'P1' and 'P2' call person 'gfg', respectively. But, there are few drawbacks in this case, as sometimes 'P1' may forget to call the person 'gfg', even after hearing the alarm, as he has a tendency to forget things, quick. Similarly, 'P2', sometimes fails to call the person 'gfg', as he is only able to hear the alarm, from a certain distance.

Calculating Conditional Probability of Events in a Bayesian Network

Find the probability that 'P1' is true (P1 has called 'gfg'), 'P2' is true (P2 has called 'gfg') when the alarm 'A' rang, but no burglary 'B' and fire 'F' has occurred.

=> $P(P1, P2, A, \sim B, \sim F)$ [where- P1, P2 & A are 'true' events and ' $\sim B$ ' & ' $\sim F$ ' are 'false' events]

[**Note:** The values mentioned below are neither calculated nor computed. They have observed values]

Burglary 'B' –

- $P(B=T) = 0.001$ ('B' is true i.e burglary has occurred)
- $P(B=F) = 0.999$ ('B' is false i.e burglary has not occurred)

Fire 'F' –

- $P(F=T) = 0.002$ ('F' is true i.e fire has occurred)
- $P(F=F) = \underline{0.998}$ ('F' is false i.e fire has not occurred)

Alarm 'A' –

B F P (A=T) P (A=F)

T T 0.95 0.05

T F 0.94 0.06

F T 0.29 0.71

F F 0.001 0.999

- The alarm 'A' node can be 'true' or 'false' (i.e may have rung or may not have rung). It has two parent nodes burglary 'B' and fire 'F' which can be 'true' or 'false' (i.e may have occurred or may not have occurred) depending upon different conditions.

Person 'P1' –

A P (P1=T) P (P1=F)

T 0.95 0.05

F 0.05 0.95

- The person 'P1' node can be 'true' or 'false' (i.e may have called the person 'gfg' or not) . It has a parent node, the alarm 'A', which can be 'true' or 'false' (i.e may have rung or may not have rung ,upon burglary 'B' or fire 'F').

Person 'P2' –

A P (P2=T) P (P2=F)

A P (P2=T) P (P2=F)

T **0.80** 0.20

F 0.01 0.99

- The person 'P2' node can be 'true' or false' (i.e may have called the person 'gfg' or not). It has a parent node, the alarm 'A', which can be 'true' or 'false' (i.e may have rung or may not have rung, upon burglary 'B' or fire 'F').

Solution: Considering the observed probabilistic scan –

With respect to the question — **P (P1, P2, A, ~B, ~F)** , we need to get the probability of 'P1'. We find it with regard to its parent node – alarm 'A'. To get the probability of 'P2', we find it with regard to its parent node — alarm 'A'.

We find the probability of alarm 'A' node with regard to '~B' & '~F' since burglary 'B' and fire 'F' are parent nodes of alarm 'A'.

From the observed probabilistic scan, we can deduce –

$$\begin{aligned}
 & \mathbf{P (P1, P2, A, \sim B, \sim F)} \\
 &= \mathbf{P (P1/A) * P (P2/A) * P (A/\sim B \sim F) * P (\sim B) * P (\sim F)} \\
 &= \mathbf{0.95 * 0.80 * 0.001 * 0.999 * 0.998} \\
 &= \mathbf{0.00075}
 \end{aligned}$$